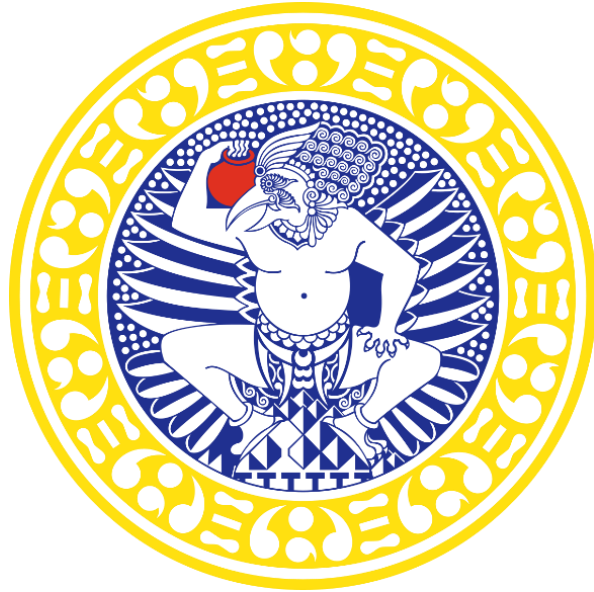


LAPORAN UTS PBL

SIAKAD Mini - RESTful API Backend



DISUSUN OLEH:

VALENTINO CHANDRA

434241045

[LINK GITHUB](#)

UNIVERSITAS AIRLANGGA

PROGRAM STUDI TEKNIK INFORMATIKA

TAHUN AJARAN 2026/2027

DAFTAR ISI

DAFTAR ISI.....	1
BAB 1 PENDAHULUAN	4
1.1 Deskripsi Proyek.....	4
1.2 Arsitektur	4
1.3 Business Rules	4
1.4 Lingkungan Pengembangan.....	5
BAB 2 SKEMA DATABASE DAN SEEDER	6
2.1 Entity Relationship Diagram.....	6
2.2 Keputusan Desain	6
2.3 Seeder.....	7
BAB 3 IMPLEMENTASI KODE PENTING.....	8
3.1 Transaction pada POST /students	8
3.2 Row Locking pada POST /enrollments	9
3.3 Batas SKS	9
3.4 Ownership di Service.....	9
3.5 Soft Delete + Login Guard.....	10
3.6 Bug Ditemukan dan Diperbaiki: UPDATE RETURNING dengan Alias.....	10
BAB 4 PENGUJIAN 10 ENDPOINT	12
4.1 Metodologi dan Lingkungan Pengujian.....	12
4.2 Ringkasan Matriks Hak Akses.....	12
4.3 Endpoint Autentikasi	13
4.3.1 POST /auth/login	13
4.3.2 GET /auth/me.....	15
4.4 Endpoint Mahasiswa.....	16
4.4.1 GET /students.....	16

4.4.2 GET /students/{id}	17
4.4.3 POST /students (khusus admin)	19
4.4.4 PUT /students/{id} (khusus admin)	20
4.4.5 DELETE /students/{id} (soft delete, khusus admin)	21
4.5 Endpoint Mata Kuliah	22
4.5.1 GET /courses	22
4.6 Endpoint Enrollment	23
4.6.1 POST /enrollments (khusus mahasiswa)	23
4.6.2 DELETE /enrollments/{id} (khusus mahasiswa, own)	25
4.7 Pengujian Business Rules (Ringkasan dari Skrip)	25
4.8 Pengujian Soft Delete (Perbaikan Bug Inti UTS)	26
4.9 Pengujian Rate Limiting	27
4.10 Rekapitulasi Hasil	28
BAB 5 BUSINESS RULES	29
5.1 Batas SKS berdasarkan IPK	29
5.2 Duplikasi Enrollment	29
5.3 Kuota Penuh	29
5.4 Ownership	30
BAB 6 UNIT TEST	31
6.1 Ringkasan Test	31
6.2 Mengapa Business Rules Diuji Terpisah	32
6.3 Contoh Test (Batas SKS)	32
BAB 7 ANALISIS	33
7.1 Mengapa Transaction Wajib di POST /students	33
7.2 Mengapa Row Locking Wajib di POST /enrollments	33
7.3 Soft Delete vs Hard Delete	33

7.4 Login Harus Memeriksa Status Bisnis.....	34
7.5 Keterbatasan yang Masih Tersisa	34
BAB 8 KESIMPULAN	36

BAB 1

PENDAHULUAN

1.1 Deskripsi Proyek

SIAKAD Mini adalah RESTful API backend untuk layanan akademik sederhana yang mengelola tiga domain utama:

- Data mahasiswa - dikelola oleh admin
- Mata kuliah - dikelola oleh admin, dibaca oleh semua role
- Kartu Rencana Studi (KRS) - mahasiswa dapat mengambil dan membatalkan mata kuliah

API terdiri dari 10 endpoint yang melayani dua peran: admin dan mahasiswa.

1.2 Arsitektur

Proyek mengikuti Clean Architecture yang konsisten dengan tugas-tugas sebelumnya:

Layer Clean Architecture	Folder	Tanggung Jawab
Entities	app/model/	Struct, DTO, dan tag validasi
Use Cases	app/service/*_rules.go	Business rules murni, tidak menyentuh HTTP atau database
Interface Adapters	app/service/*_service.go, app/repository/, helper/	Orchestration + HTTP + SQL
Frameworks & Drivers	config/, database/, middleware/, route/, main.go	Fiber, PostgreSQL, logger, DI

Pemisahan ini memungkinkan business rules diuji tanpa server (Bab 6) dan repository dapat diganti tanpa mengubah service (dependency inversion).

1.3 Business Rules

Empat aturan bisnis yang diimplementasikan sesuai spesifikasi:

1. Batas SKS berdasarkan IPK terakhir:
 - $IPK \geq 3.00 \rightarrow$ maksimal 24 SKS
 - $2.50 \leq IPK < 3.00 \rightarrow$ maksimal 21 SKS
 - $IPK < 2.50 \rightarrow$ maksimal 18 SKS

2. Tidak boleh duplikat, mahasiswa tidak boleh mengambil mata kuliah yang sama dua kali pada tahun akademik yang sama.
3. Kuota terbatas, mata kuliah dengan kuota penuh tidak dapat diambil.
4. Ownership, mahasiswa hanya dapat mengakses dan memodifikasi KRS miliknya sendiri.

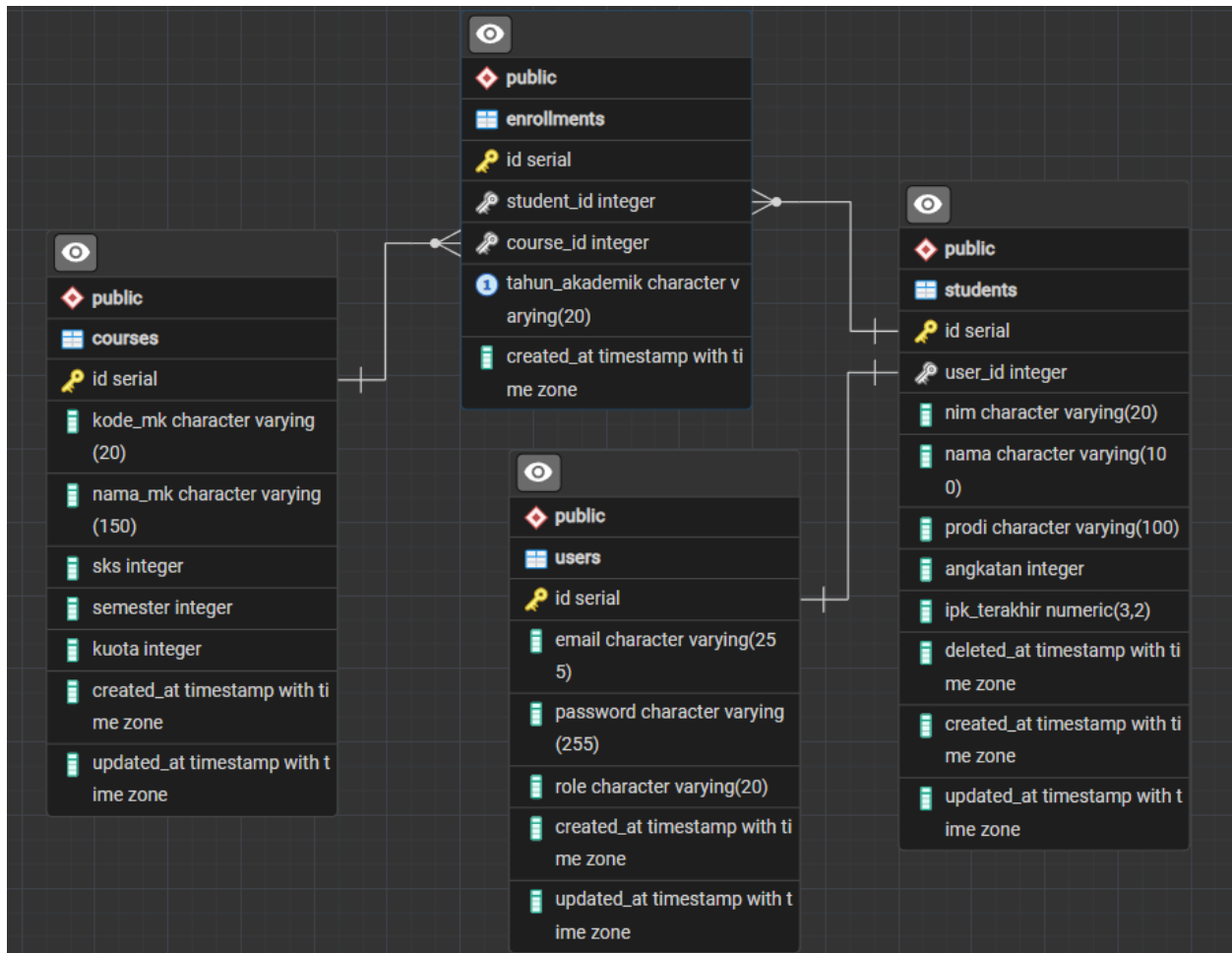
1.4 Lingkungan Pengembangan

Komponen	Spesifikasi
Bahasa	Go 1.27
Framework	Fiber v2
Database	PostgreSQL 16
Library	pgx/v5, golang-jwt/jwt/v5, golang.org/x/crypto/bcrypt, go-playground/validator/v10, lumberjack.v2
Shell pengujian	PowerShell + curl.exe
Testing	go test ./... -v, skrip e2e test-uts.ps 1
Base URL	http://localhost:3000/api/v1
Autentikasi	JWT Bearer, expires_in = 900 detik
Klien manual	Postman (environment SIAKAD Local)

BAB 2

SKEMA DATABASE DAN SEEDER

2.1 Entity Relationship Diagram



2.2 Keputusan Desain

1. **students.user_id UNIQUE** Menegakkan relasi 1-1. Satu akun hanya boleh menjadi satu mahasiswa. Ini dijaga database, bukan kode aplikasi.
2. **enrollments UNIQUE (student_id, course_id, tahun_akademik)**, Business rule #2 diterapkan di level database. Ini menerapkan defense in depth: meskipun ada bug di kode yang lupa memeriksa duplikasi, database tetap menolak. Aturan penting harus dijaga di lebih dari satu lapis.
3. **enrollments.course_id ON DELETE RESTRICT**, Mata kuliah yang masih diambil tidak boleh dihapus. Mencegah data yatim.

4. **students.deleted_at (soft delete)**, Mahasiswa yang dihapus tetap ada di database untuk audit, tapi tidak muncul di list dan tidak dapat login.
5. **Index parsial WHERE deleted_at IS NULL**, Hanya mengindeks mahasiswa aktif. Query GET /students lebih cepat karena index lebih kecil.
6. **NUMERIC(3,2) untuk IPK**, Angka akademik harus presisi. FLOAT tidak dijamin bisa menghasilkan 3.4499999 untuk 3.45. NUMERIC menyimpan desimal.

2.3 Seeder

Seeder membuat:

Role	Jumlah	Kredensial
Admin	1	admin@siakad.test / admin12345
Mahasiswa	20	{NIM}@student.siakad.test / password = NIM
Mata kuliah	10	Tersebar di semester 1–6

Password di-hash dengan `crypt(password, gen_salt('bf', 12))` dari ekstensi PostgreSQL `pgcrypto`. Ini menghasilkan hash `bcrypt cost 12` yang kompatibel dengan verifikasi `bcrypt` di Go, dibuktikan dengan login mahasiswa yang sukses (`password = NIM`).

```
PS D:\DEV\go\backend-lanjut\UTS> psql -U postgres -d siakad_mini -c "SELECT role, COUNT(*) FROM users GROUP BY role;"
Password for user postgres:

 role | count
-----+-----
 admin |     1
 mahasiswa |    21
(2 rows)

PS D:\DEV\go\backend-lanjut\UTS> psql -U postgres -d siakad_mini -c "SELECT email, LEFT(password, 7) FROM users LIMIT 3;"
Password for user postgres:

 email | left
-----+-----
 admin@siakad.test | $2a$12$
 187221000001@student.siakad.test | $2a$12$
 187221000002@student.siakad.test | $2a$12$
(3 rows)

PS D:\DEV\go\backend-lanjut\UTS> psql -U postgres -d siakad_mini -c "SELECT COUNT(*) FROM courses;"
Password for user postgres:

 count
-----
    10
(1 row)
```

BAB 3

IMPLEMENTASI KODE PENTING

3.1 Transaction pada POST /students

Pembuatan mahasiswa baru memerlukan dua insert, ke users dan ke students. Keduanya harus atomik:

```
func (r *studentPostgresRepository) CreateWithUser(
    ctx context.Context, email, passwordHash string, s model.Student,
) (model.Student, error) {
    tx, err := r.pool.Begin(ctx)
    if err != nil {
        return model.Student{}, fmt.Errorf("memulai transaksi: %w", err)
    }
    defer tx.Rollback(ctx) // no-op bila sudah commit

    var userID int
    err = tx.QueryRow(ctx,
        `INSERT INTO users (email, password, role) VALUES ($1, $2, 'mahasiswa')
        RETURNING id`,
        email, passwordHash,
    ).Scan(&userID)
    if err != nil {
        if isUniqueViolation(err) {
            return model.Student{}, ErrDuplicate
        }
        return model.Student{}, fmt.Errorf("menyimpan user: %w", err)
    }

    err = tx.QueryRow(ctx,
        `INSERT INTO students (user_id, nim, nama, prodi, angkatan, ipk_terakhir)
        VALUES ($1, $2, $3, $4, $5, $6)
        RETURNING id, created_at, updated_at`,
        userID, s.NIM, s>Nama, s.Prodi, s.Angkatan, s.IPKTerakhir,
    ).Scan(&s.ID, &s.CreatedAt, &s.UpdatedAt)
    if err != nil {
        if isUniqueViolation(err) {
            return model.Student{}, ErrDuplicate
        }
        return model.Student{}, fmt.Errorf("menyimpan student: %w", err)
    }
    s.UserID = userID

    if err := tx.Commit(ctx); err != nil {
        return model.Student{}, fmt.Errorf("commit transaksi: %w", err)
    }
    return s, nil
}
```

Jika salah satu insert gagal, defer tx.Rollback(ctx) akan mengembalikan database ke keadaan semula, tidak ada user tanpa student, tidak ada student tanpa user.

3.2 Row Locking pada POST /enrollments

Ini bagian paling kritis. Dua mahasiswa yang bersamaan mengambil mata kuliah terakhir harus di-serialisasi:

```
func (r *enrollmentPostgresRepository) LockCourseForUpdate(
    ctx context.Context, tx pgx.Tx, courseID int,
) (model.Course, error) {
    var c model.Course
    err := tx.QueryRow(ctx,
        `SELECT id, kode_mk, nama_mk, sks, semester, kuota, created_at, updated_at
        FROM courses WHERE id = $1 FOR UPDATE`,
        courseID,
    ).Scan(&c.ID, &c.KodeMK, &c>NamaMK, &c.SKS, &c.Semester,
        &c.Kuota, &c.CreatedAt, &c.UpdatedAt)
    // ...
    return c, nil
}
```

SELECT ... FOR UPDATE mengunci baris courses sampai transaksi commit atau rollback. Request kedua akan menunggu, bukan membaca data lama. Inilah yang mencegah race condition pada cek kuota.

3.3 Batas SKS

```
func BatasSKS(ipk float64) int {
    switch {
    case ipk >= 3.00:
        return 24
    case ipk >= 2.50:
        return 21
    default:
        return 18
    }
}

func CanEnrollSKS(totalSKS, sksBaru, batas int) error {
    if totalSKS+sksBaru > batas {
        sisa := batas - totalSKS
        if sisa < 0 {
            sisa = 0
        }
        return fmt.Errorf(
            "total SKS melebihi batas %d. Sisa SKS yang dapat diambil: %d",
            batas, sisa)
    }
    return nil
}
```

Fungsi murni. Tidak menyentuh database, tidak menyentuh Fiber. Mudah diuji (Bab 6) dan mudah dijelaskan.

3.4 Ownership di Service

```
if current.Role == "mahasiswa" {
    own, err := s.students.FindByUserID(ctx, current.UserID)
    if err != nil || own.ID != student.ID {
        return helper.Forbidden("Anda tidak berhak mengakses data mahasiswa lain")
    }
}
```

Ownership tidak dapat dipindahkan ke middleware: middleware tidak tahu `id` merujuk ke baris siapa tanpa query database. Menempatkannya di middleware melanggar Clean Architecture (middleware jadi tahu struktur data) dan menggandakan query. Karena itu keputusan ini diambil di service, setelah data di-fetch.

3.5 Soft Delete + Login Guard

Soft delete menggunakan kolom `deleted_at`:

```
func (r *studentPostgresRepository) SoftDelete(ctx context.Context, id int) error {
    tag, err := r.pool.Exec(ctx,
        `UPDATE students SET deleted_at = NOW()
        WHERE id = $1 AND deleted_at IS NULL`,
        id,
    )
    // ...
}
```

Query `WHERE deleted_at IS NULL` membuat operasi idempotent, pemanggilan kedua tidak mengubah `deleted_at` yang sudah ada, dan mengembalikan `ErrNotFound`.

Untuk menjaga konsistensi, login wajib memeriksa apakah entitas bisnis mahasiswa masih aktif:

```
if user.Role == "mahasiswa" {
    if _, err := s.students.FindByUserID(ctx, user.ID); err != nil {
        return helper.Unauthorized("Akun Anda sudah tidak aktif")
    }
}
```

Tanpa pemeriksaan ini, mahasiswa yang dihapus dapat terus login, memisahkan "identitas valid" (user) dari "entitas aktif" (student) adalah kesalahan desain yang umum di sistem nyata.

3.6 Bug Ditemukan dan Diperbaiki: UPDATE RETURNING dengan Alias

Gejala: Endpoint `PUT /students/{id}` mengembalikan 500 meskipun data valid.

Log server:

```
"error": "memperbarui student: ERROR: missing FROM-clause entry for table \"s\" (SQLSTATE 42P01)"
```

Akar masalah: Konstanta `studentColumns` didefinisikan dengan alias `s`:

```
const studentColumns = `s.id, s.user_id, s.nim, ...`
```

Ini benar untuk `SELECT ... FROM students s`, tetapi salah untuk `UPDATE ... RETURNING`. PostgreSQL tidak mengenali alias `s` di klausa `UPDATE` karena `UPDATE` tidak menerima alias.

Perbaikan: Konstanta terpisah tanpa alias:

```
const studentColumnsBare = `id, user_id, nim, nama, prodi, angkatan,
    ipk_terakhir, deleted_at, created_at, updated_at`
```

Method Update diubah menjadi:

```
RETURNING `+studentColumnsBare,
```

Bukti: setelah perbaikan, PUT /students/1 oleh admin mengembalikan 200 OK (lihat Bab 4).

Pelajaran: Alias tabel di SELECT tidak berlaku universal. RETURNING pada INSERT/UPDATE/DELETE hanya menerima nama kolom telanjang atau ekspresi, bukan alias. Ini konsisten dengan spesifikasi SQL: alias didefinisikan di klausa FROM, dan UPDATE tidak memiliki klausa FROM dalam bentuk yang sama.

BAB 4

PENGUJIAN 10 ENDPOINT

4.1 Metodologi dan Lingkungan Pengujian

Pengujian dilakukan pada dua lapis:

1. Pengujian fungsional end-to-end melalui skrip otomatis `scripts/test-uts.ps1` yang menjalankan 31 assertion terhadap API yang berjalan di `http://localhost:3000/api/v1`. Skrip ini bersifat idempoten: sebelum dijalankan, ia membersihkan enrollment dan sisa data mahasiswa uji dari run sebelumnya sehingga hasil konsisten di setiap eksekusi.
2. Pengujian manual melalui Postman untuk memverifikasi bentuk respons JSON, header, dan status code secara visual, hasilnya dilampirkan sebagai screenshot.

Akun uji:

Role	Email	Password
Admin	admin@siakad.test	admin12345
Mahasiswa	187221000001@student.siakad.test	187221000001

4.2 Ringkasan Matriks Hak Akses

Tabel berikut merangkum hasil pengujian terhadap 10 endpoint utama. Kolom angka adalah HTTP status code yang diterima.

#	Endpoint	Admin	Mahasiswa	Publik (tanpa token)
1	POST /auth/login	200	200	200
2	GET /auth/me	200	200	401
3	GET /students	200	403	401
4	POST /students	201	403	401
5a	GET /students/{id} (self)	200	200	401
5b	GET /students/{id} (other)	200	403	401
6	PUT /students/{id}	200	403	401
7	DELETE /students/{id}	204	403	401
8	GET /courses	200	200	401
9	POST /enrollments	403	201	401
10	DELETE /enrollments/{id} (own)	403	204	401

Perhatikan pola:

- Endpoint publik hanya POST /auth/login.
- Endpoint baca data master (/courses) boleh diakses kedua role.
- Endpoint mutasi mahasiswa (POST/PUT/DELETE /students) hanya admin.
- Endpoint enrollment hanya mahasiswa (admin sengaja ditolak 403, sesuai aturan bisnis: KRS adalah milik mahasiswa).

0. RESET (hapus enrollment + mahasiswa uji)				4. BUSINESS RULES – ENROLLMENT			
[OK] Data mahasiswa uji dibersihkan				[OK] POST /enrollments (valid pertama kali)	harap=201	dapat=201	
				[OK] POST /enrollments (duplikat -> 400)	harap=400	dapat=400	
				[OK] POST /enrollments (tahun akademik salah -> 422)	harap=422	dapat=422	
				[OK] POST /enrollments (course_id tidak ada -> 422)	harap=422	dapat=422	
				[OK] POST /enrollments (admin -> 403)	harap=403	dapat=403	
1. LOGIN				Detail mahasiswa setelah 1 enrollment:			
Mahasiswa login: user_id=2, student_id=1				NIM = 187221000001			
Admin login -> 200, role = admin				Nama = Rina Putri Updated			
Mahasiswa login -> 200, role = mahasiswa				IPK = 3.55			
[OK] Login password salah	harap=401	dapat=401		Total SKS = 3			
[OK] Login email format salah	harap=422	dapat=422		Batas SKS = 24			
				Mata kuliah = 1			
2. AUTHORIZATION MATRIX				5. DELETE /enrollments			
[OK] GET /auth/me (admin)	harap=200	dapat=200					
[OK] GET /auth/me (mahasiswa)	harap=200	dapat=200		[OK] DELETE /enrollments (admin -> 403)	harap=403	dapat=403	
[OK] GET /auth/me (no token)	harap=401	dapat=401		[OK] DELETE /enrollments (mahasiswa milik sendiri -> 204)	harap=204	dapat=204	
[OK] GET /students (admin)	harap=200	dapat=200		[OK] DELETE /enrollments (id sama lagi -> 404)	harap=404	dapat=404	
[OK] GET /students (mahasiswa -> 403)	harap=403	dapat=403		6. SOFT DELETE STUDENT			
[OK] GET /students/1 (mahasiswa self)	harap=200	dapat=200					
[OK] GET /students/2 (mahasiswa other -> 403)	harap=403	dapat=403		[OK] POST /students (admin buat baru -> 201)	harap=201	dapat=201	
[OK] GET /students/1 (admin)	harap=200	dapat=200		Mahasiswa baru ID = 24			
[OK] GET /students/99999 (not found -> 404)	harap=404	dapat=404		[OK] DELETE /students/24 (soft delete -> 204)	harap=204	dapat=204	
[OK] PUT /students/1 (admin)	harap=200	dapat=200		[OK] Login mahasiswa terhapus -> 401	harap=401	dapat=401	
[OK] PUT /students/1 (mahasiswa -> 403)	harap=403	dapat=403		[OK] Mahasiswa terhapus tidak muncul di list	harap=0	dapat=0	
3. GET /courses				[OK] GET /students/24 (soft deleted -> 404)	harap=404	dapat=404	
[OK] GET /courses (admin)	harap=200	dapat=200		7. RATE LIMITING (6x login gagal)			
[OK] GET /courses (mahasiswa)	harap=200	dapat=200		Hasil: 401, 401, 401, 401, 429, 429			
[OK] GET /courses (no token -> 401)	harap=401	dapat=401		[OK] Rate limiter bekerja (2 request di-429)			
Daftar 3 mata kuliah pertama:				SELESAI			
				Semua pengujian selesai.			
id kode_mk nama_mk sks kuota terisi sisa_kuota							
1 IF1234 Pemrograman Dasar 3 40 0 40							
2 IF1235 Logika Informatika 3 40 0 40							
3 IF2101 Struktur Data 3 35 0 35							

4.3 Endpoint Autentikasi

4.3.1 POST /auth/login

Request (Postman: Body → raw → JSON):

```
POST http://localhost:3000/api/v1/auth/login
Content-Type: application/json

{
  "email": "admin@siakad.test",
  "password": "admin12345"
}
```

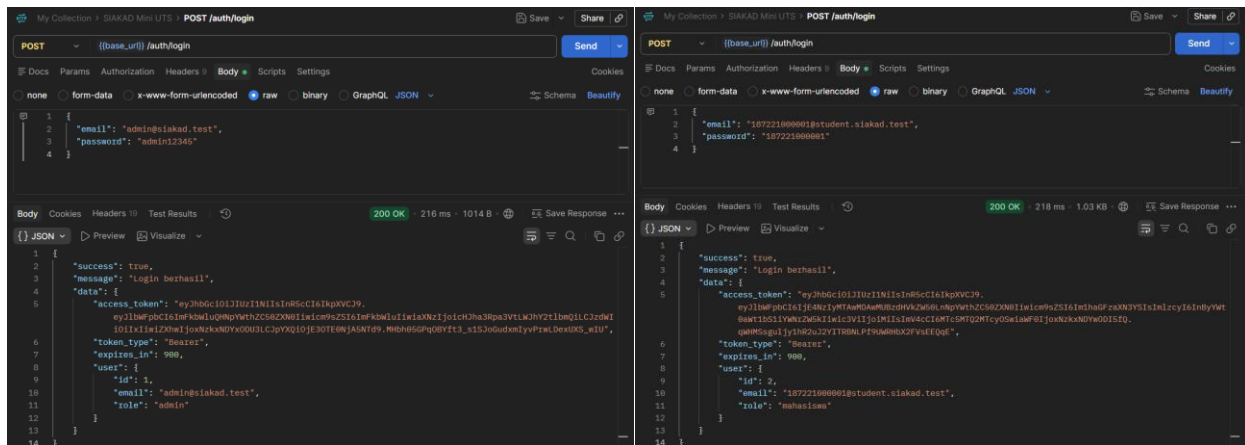
Response 200 OK:

```
{
```

```

"success": true,
"message": "Login berhasil",
"data": {
  "access_token": "eyJhbGciOi...",
  "token_type": "Bearer",
  "expires_in": 900,
  "user": { "id": 1, "email": "admin@siakad.test", "role": "admin" }
}
}

```



Variasi pengujian yang dilakukan:

Skenario	Request	Status	Body
Password salah	{"email": "admin@siakad.test", "password": "salahbanget"}	401	{"success": false, "code": "UNAUTHORIZED", "message": "email atau password salah"}
Email tidak terdaftar	{"email": "tidak@ada.test", "password": "admin12345"}	401	Pesan identik dengan password salah
Format email salah	{"email": "bukan-email", "password": "admin12345"}	422	{"errors": {"email": "Format email tidak valid"}}
Password kurang dari 8 karakter	{"email": "admin@siakad.test", "password": "abc"}	422	{"errors": {"password": "minimal 8 karakter"}}

Catatan penting (Anti Timing Attack):

Pesan error untuk "password salah" dan "email tidak terdaftar" identik. Ini disengaja: jika pesannya berbeda, penyerang dapat memetakan email mana yang terdaftar (user enumeration). Selain itu, ketika email tidak ditemukan, server tetap menjalankan bcrypt dengan hash dummy (VerifyDummyPassword) agar waktu tanggap serupa dengan kasus password salah, mencegah kebocoran melalui analisis timing.

4.3.2 GET /auth/me

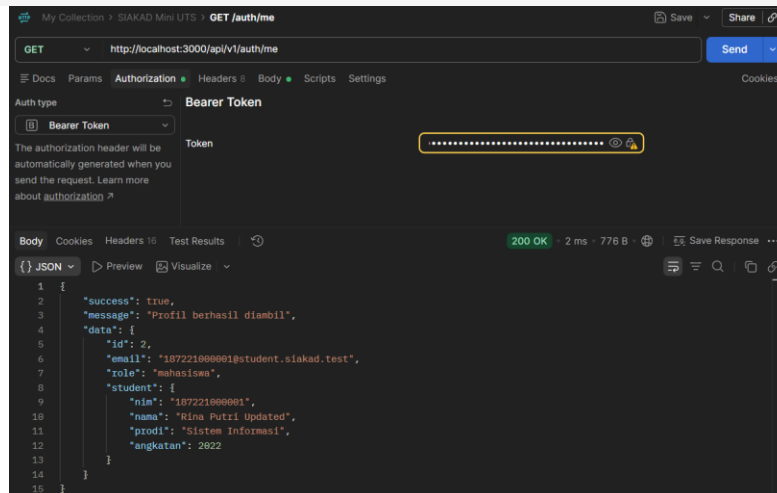
Mengembalikan identitas pengguna yang sedang login. Untuk role mahasiswa, field student disertakan; untuk role lain, field di-omitempty.

Request:

```
GET http://localhost:3000/api/v1/auth/me
Authorization: Bearer <TOKEN_MHS>
```

Response 200 OK:

```
{
  "success": true,
  "message": "Profil berhasil diambil",
  "data": {
    "id": 2,
    "email": "187221000001@student.siakad.test",
    "role": "mahasiswa",
    "student": {
      "nim": "187221000001",
      "nama": "Rina Putri",
      "prodi": "Sistem Informasi",
      "angkatan": 2022
    }
  }
}
```



Perhatikan: student hanya muncul untuk role mahasiswa. Untuk admin, field ini dihilangkan (omitempty).

Skenario error:

Skenario	Status	Body
Tanpa header Authorization	401	{"code":"UNAUTHORIZED","message":"header Authorization tidak ada atau salah bentuk"}
Token invalid (satu huruf diubah)	401	{"message":"access token tidak valid"}
Token kedaluwarsa	401	{"message":"access token kedaluwarsa"}

4.4 Endpoint Mahasiswa

4.4.1 GET /students

Daftar mahasiswa dengan paginasi, filter, dan sorting.

Query parameter yang didukung:

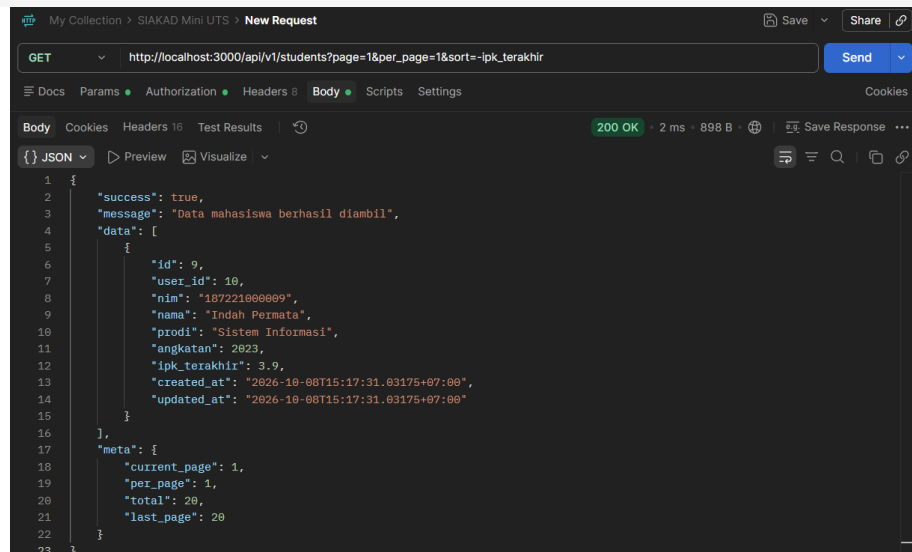
Parameter	Nilai	Default
page	integer ≥ 1	1
per_page	1–50	10
prodi	string	-
angkatan	tahun	-
search	substring nama atau nim	-
sort	nama, -nama, ipk_terakhir, -ipk_terakhir	nama

Request:

```
GET http://localhost:3000/api/v1/students?page=1&per_page=5&sort=-ipk_terakhir
Authorization: Bearer <TOKEN_ADMIN>
```

Response 200 OK:

```
{
  "success": true,
  "message": "Data mahasiswa berhasil diambil",
  "data": [
    {
      "id": 9,
      "nim": "187221000009",
      "nama": "Indah Permata",
      "prodi": "Sistem Informasi",
      "angkatan": 2023,
      "ipk_terakhir": 3.90
    }
  ],
  "meta": {
    "current_page": 1,
    "per_page": 5,
    "total": 20,
    "last_page": 4
  }
}
```



page1 biar kelihatan "meta"-nya

Variasi filter yang diuji:

Skenario	URL	Hasil
Filter prodi	?prodi=Sistem Informasi	Hanya mahasiswa prodi Sistem Informasi
Filter angkatan	?angkatan=2023	Hanya mahasiswa angkatan 2023
Search	?search=Rina	Mahasiswa dengan nama mengandung "Rina"
Sort IPK menurun	?sort=-ipk_terakhir	IPK tertinggi di atas
Pagination	?page=2&per_page=5	Halaman 2 dari 4

Skenario error:

Skenario	Status	Body
Mahasiswa mengakses	403	{"code":"FORBIDDEN","message":"role Anda tidak berhak mengakses endpoint ini"}
Tanpa token	401	{"message":"header Authorization tidak ada atau salah bentuk"}

4.4.2 GET /students/{id}

Endpoint agregat: mengembalikan profil mahasiswa sekaligus KRS dan kalkulasi SKS, mengurangi round-trip dari frontend.

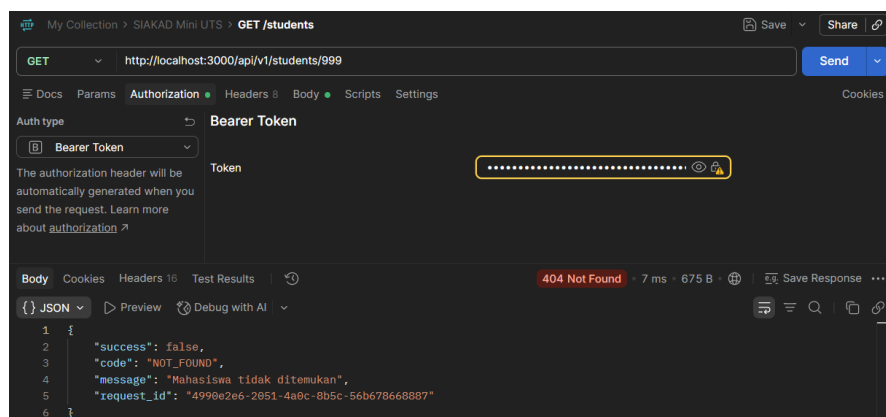
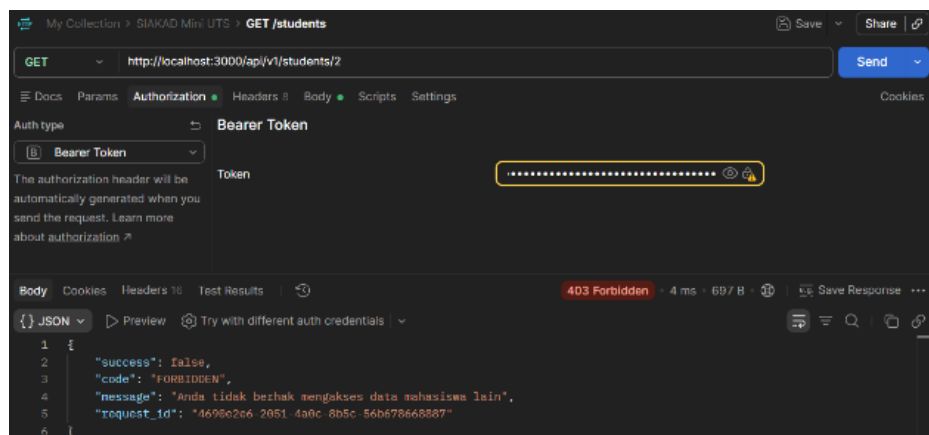
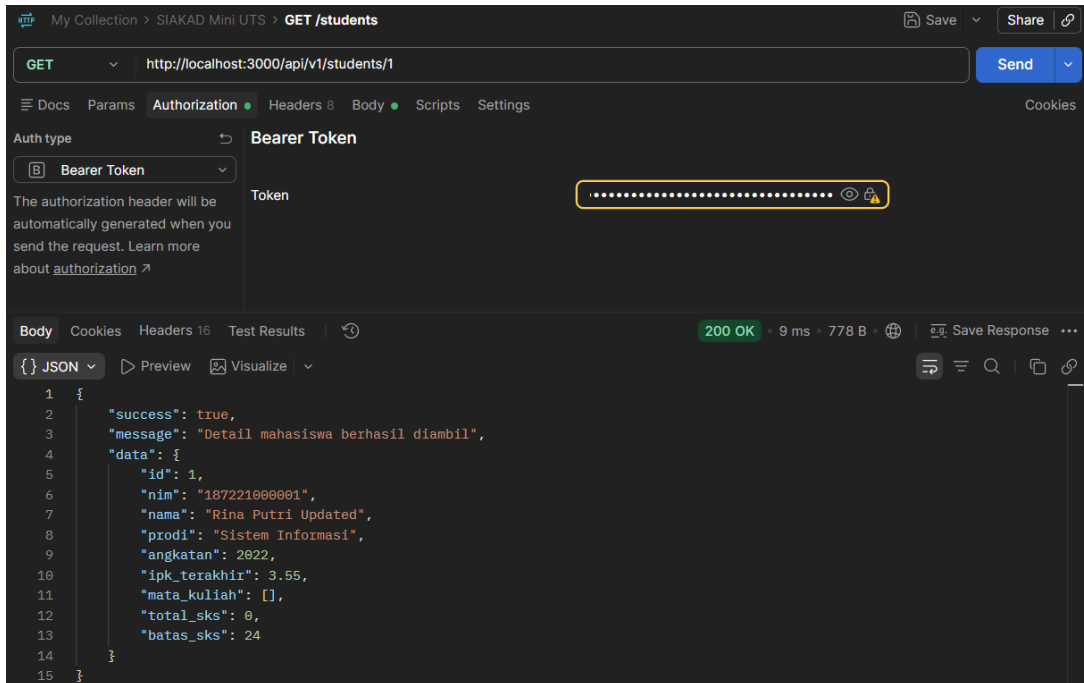
Request:

```
GET http://localhost:3000/api/v1/students/1
Authorization: Bearer <TOKEN_MHS>
```

Response 200 OK:

```
{
  "success": true,
  "message": "Detail mahasiswa berhasil diambil",
  "data": {
    "id": 1, "nim": "187221000001", "nama": "Rina Putri Updated",
    "prodi": "Sistem Informasi", "angkatan": 2022, "ipk_terakhir": 3.55,
    "mata_kuliah": [
      {
        "enrollment_id": 2, "course_id": 1,
        "kode_mk": "IF1234", "nama_mk": "Pemrograman Dasar",
        "sks": 3, "semester": 1, "tahun_akademik": "2026/2027-Ganjil"
      }
    ],
    "total_sks": 3,
    "batas_sks": 24
  }
}
```

Aturan kepemilikan: mahasiswa hanya boleh mengakses dirinya sendiri (student_id cocok dengan user_id pada token). Mengakses ID mahasiswa lain → **403**; ID yang tidak ada → **404**; tanpa token → **401**.



Tiga skenario kepemilikan:

Skenario	URL	Status	Keterangan
Mahasiswa akses dirinya	/students/1 (dengan token MHS-1)	200	student_id cocok dengan user_id
Mahasiswa akses orang lain	/students/2 (dengan token MHS-1)	403	{"message": "Anda tidak berhak mengakses data mahasiswa lain"}
ID tidak ada	/students/99999	404	{"code": "NOT_FOUND", "message": "Mahasiswa tidak ditemukan"}
ID sudah soft delete	/students/21	404	Sama, baris tidak muncul karena filter deleted_at IS NULL

4.4.3 POST /students (khusus admin)

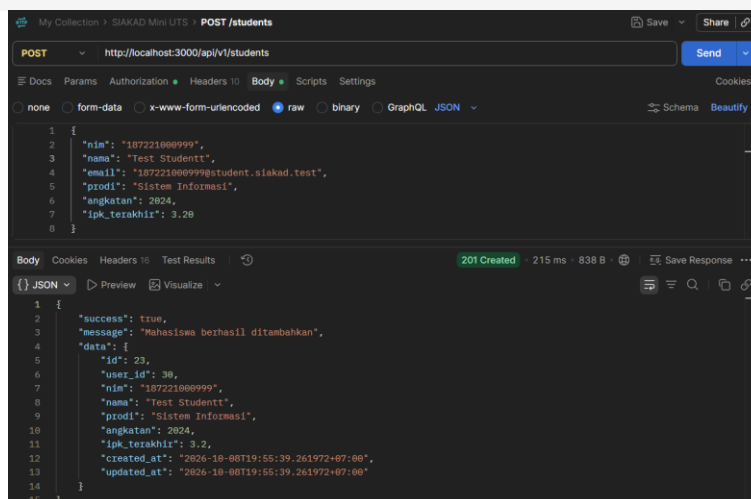
Request:

```
POST http://localhost:3000/api/v1/students
Authorization: Bearer <TOKEN_ADMIN>
Content-Type: application/json

{
  "nim": "187221000999",
  "nama": "Test Studentt",
  "email": "187221000999@student.siakad.test",
  "prodi": "Sistem Informasi",
  "angkatan": 2024,
  "ipk_terakhir": 3.20
}
```

Response 201 Created:

```
{
  "success": true,
  "message": "Mahasiswa berhasil dibuat",
  "data": { "id": 23, "nim": "187221000999", "...": "..." }
}
```

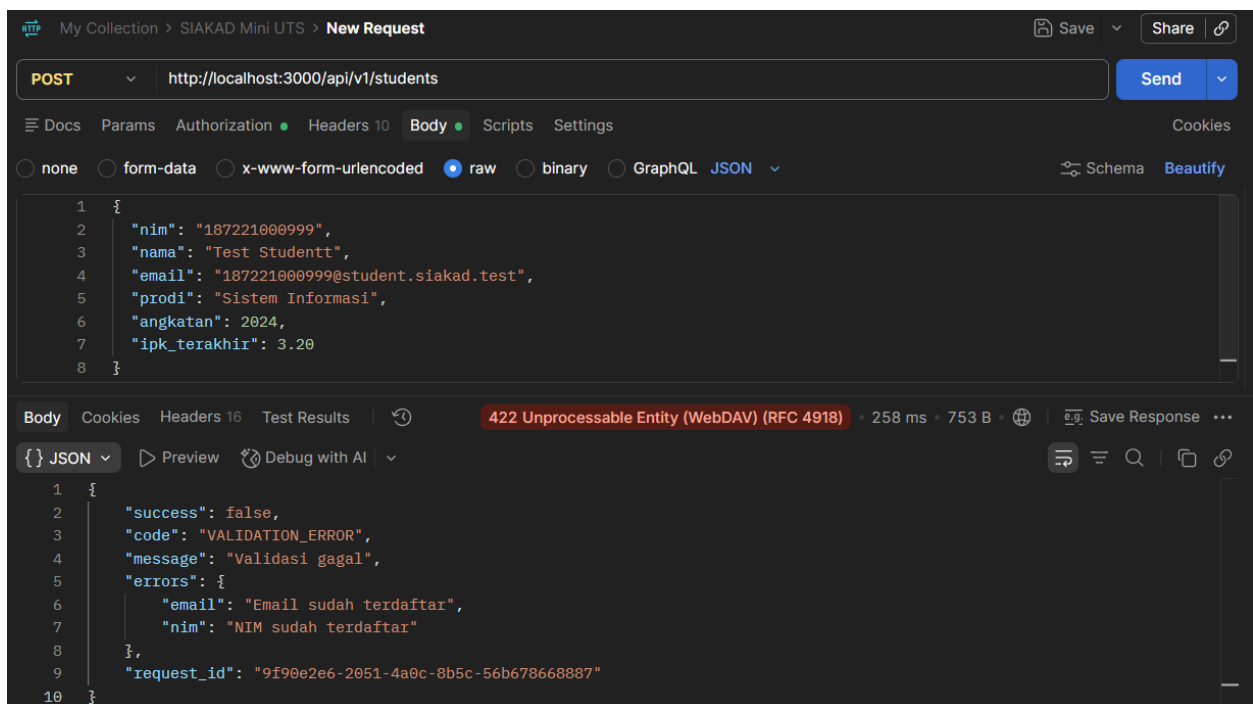


Proses di balik layar:

1. Hash password awal = NIM.
2. Transaction dimulai → insert ke users (role mahasiswa) → insert ke students → commit.
3. Jika salah satu insert gagal → rollback, tidak ada data parsial.

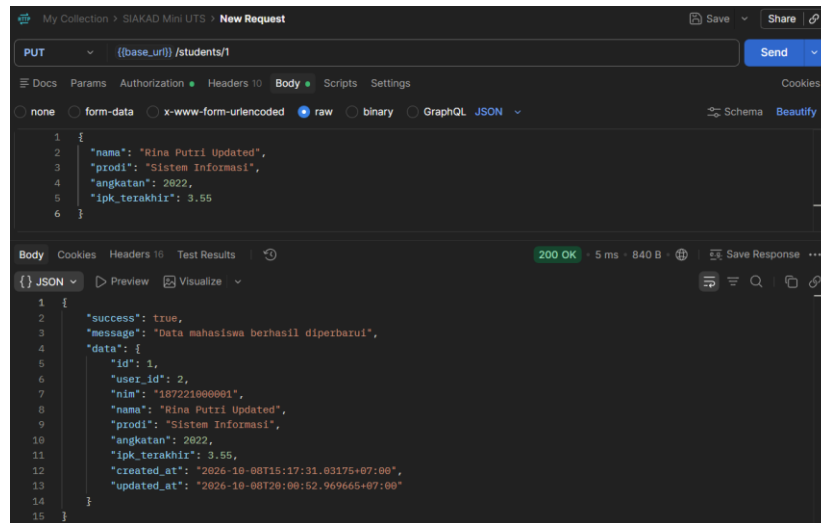
Skenario error:

Skenario	Status	Body
Mahasiswa mengakses	403	{"code":"FORBIDDEN"}
NIM sudah terdaftar	422	{"errors":{"nim":"NIM sudah terdaftar"}}
Email sudah terdaftar	422	{"errors":{"email":"Email sudah terdaftar"}}
NIM bukan 12 digit	422	{"errors":{"nim":"harus tepat 12 karakter"}}
IPK di luar 0–4	422	{"errors":{"ipk_terakhir":"..."}}



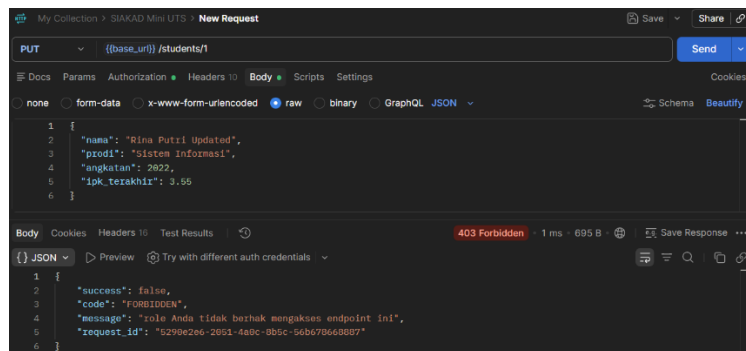
4.4.4 PUT /students/{id} (khusus admin)

Body yang diterima sama seperti POST (semua field opsional, hanya yang dikirim yang di-update). Mahasiswa yang mencoba → **403**.



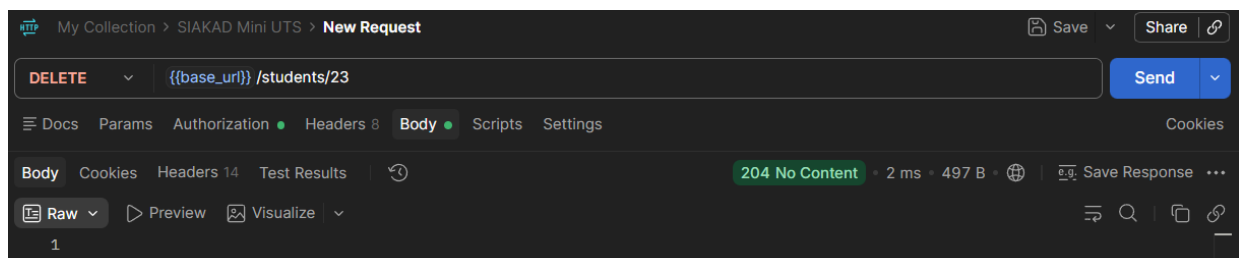
Skenario error:

Skenario	Status
Mahasiswa mengakses	403
ID tidak ada	404
Field nama kosong	422
angkatan di luar 2000–2100	422



4.4.5 DELETE /students/{id} (soft delete, khusus admin)

Response 204 No Content.



Efek setelah soft delete:

1. GET /students — mahasiswa ID 21 tidak muncul lagi (difilter deleted_at IS NULL).
2. GET /students/21 → **404**.
3. Login mahasiswa tersebut → **401** "Akun Anda sudah tidak aktif", perbaikan bug inti UTS (lihat BAB 4.8).

Skenario error:

Skenario	Status
Mahasiswa mengakses	403
ID tidak ada	404
ID sudah di-soft delete sebelumnya	404 (idempotent)

4.5 Endpoint Mata Kuliah

4.5.1 GET /courses

Query parameter: available=true (hanya menampilkan mata kuliah dengan sisa_kuota > 0).

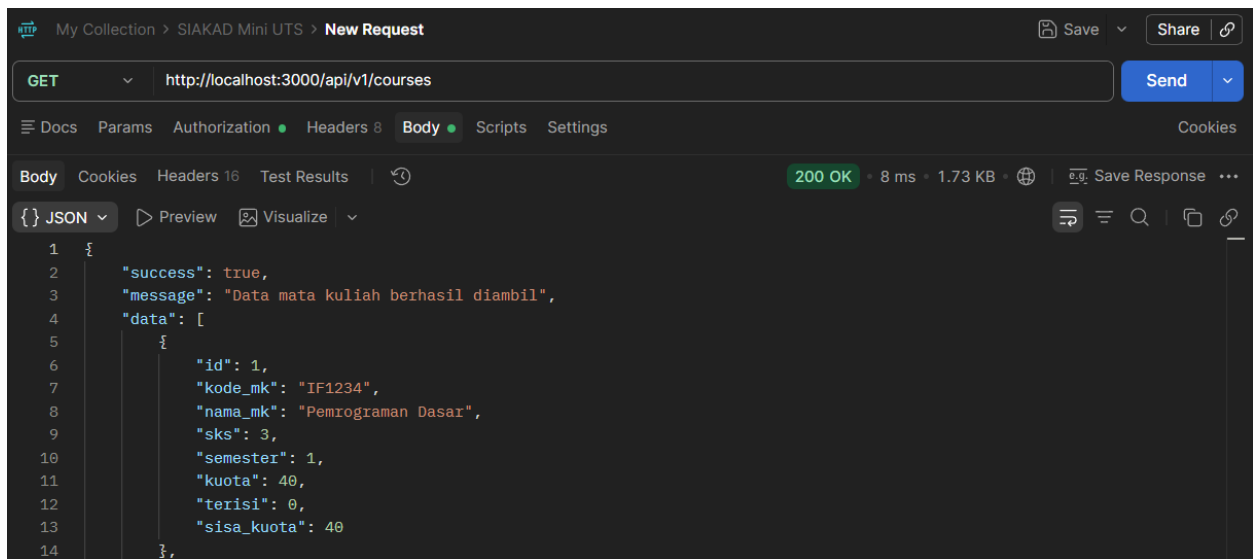
Request:

```
GET http://localhost:3000/api/v1/courses
Authorization: Bearer <TOKEN_MHS>
```

Response 200 OK:

```
{
  "success": true,
  "message": "Data mata kuliah berhasil diambil",
  "data": [
    {
      "id": 1, "kode_mk": "IF1234", "nama_mk": "Pemrograman Dasar",
      "sks": 3, "semester": 1, "kuota": 40,
      "terisi": 1, "sisa_kuota": 39
    }
  ]
}
```

Field terisi dan sisa_kuota dihitung dari subquery ke tabel enrollments, bukan kolom tersimpan. Ini penting untuk konsistensi: nilai selalu benar meskipun enrollment ditambah atau dihapus.



Skenario yang diuji:

Skenario	URL	Keterangan
Filter semester	?semester=1	Hanya MK semester 1
Filter available	?available=true	MK dengan sisa kuota > 0
Search	?search=IF	Semua MK dengan kode mulai IF
Tanpa token	-	401

Catatan desain (terisi dan sisa_kuota):

Field terisi dan sisa_kuota dihitung dari subquery ke tabel enrollments, bukan kolom tersimpan di courses. Ini penting untuk konsistensi: nilainya selalu benar meskipun enrollment ditambah atau dihapus, tanpa risiko kolom cache yang basi.

4.6 Endpoint Enrollment

4.6.1 POST /enrollments (khusus mahasiswa)

Request:

```

POST
Authorization: Bearer <TOKEN_MHS>
Content-Type: application/json

{
  "course_id": 1,
  "tahun_akademik": "2026/2027-Ganjil"
}

```

Response 201 Created.

Proses di balik layar (di dalam transaction):

1. Lock course, SELECT ... FROM courses WHERE id = \$1 FOR UPDATE.
2. Cek duplikasi, SELECT EXISTS(...) pada (student_id, course_id, tahun_akademik).
3. Cek kuota, SELECT COUNT(*) dari enrollments untuk course yang sama di tahun yang sama; bandingkan dengan courses.kuota.
4. Cek batas SKS, SUM(sks) dari semua enrollment mahasiswa di tahun yang sama; bandingkan dengan BatasSKS(ipk_terakhir).
5. Insert enrollment.
6. Commit.

Aturan bisnis yang diuji:

Skenario	Status	Keterangan
Enrollment valid pertama	201	-
Duplikat (course + tahun yang sama)	409	Constraint unik di database
Format tahun_akademik salah	422	Harus berpola YYYY/YYYY-Semester
course_id tidak ada	422	Validasi relasi
Kuota penuh	422	Dicek melalui sisa_kuota
Total SKS melebihi batas_sks	422	batas_sks = 24
Admin mencoba enroll	403	Hanya mahasiswa

```
POST http://localhost:3000/api/v1/enrollments

{
  "course_id": 1,
  "tahun_akademik": "2026/2027-Ganjil"
}
```

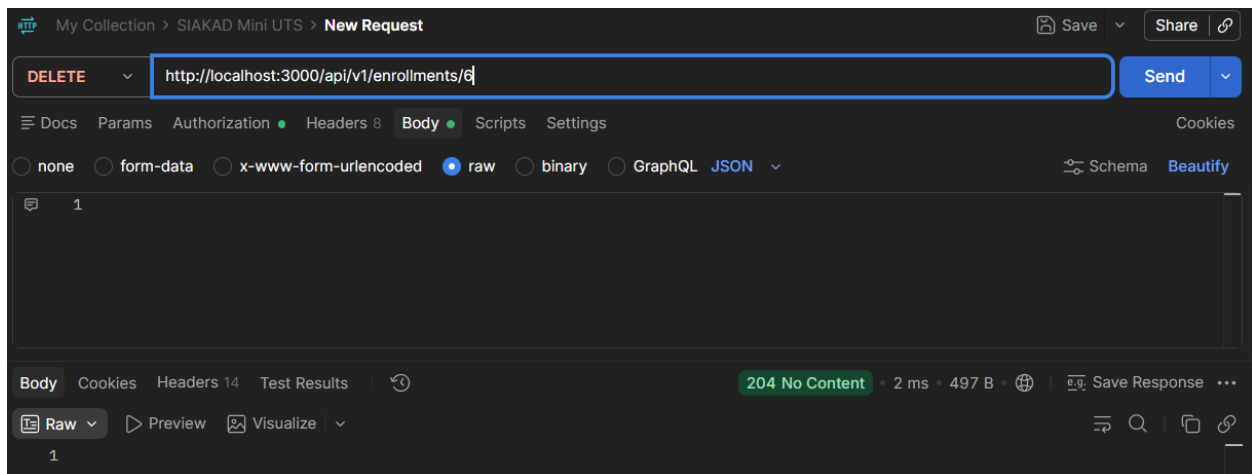
```
{
  "success": true,
  "message": "Mata kuliah berhasil diambil",
  "data": {
    "id": 6,
    "course_id": 1,
    "kode_mk": "IF1234",
    "nama_mk": "Pemrograman Dasar",
    "sks": 3,
    "tahun_akademik": "2026/2027-Ganjil",
    "created_at": "2026-10-08T20:09:34+07:00"
  }
}
```

4.6.2 DELETE /enrollments/{id} (khusus mahasiswa, own)

Mahasiswa hanya boleh menghapus enrollment miliknya. Percobaan kedua dengan ID yang sama → **404** (sudah tidak ada).

Response 204 No Content.

Efek: kuota mata kuliah kembali bertambah karena COUNT(*) di subquery GET /courses otomatis berkurang.



Skenario error:

Skenario	Status	Keterangan
Hapus ID yang sama lagi	404	Sudah tidak ada
Admin mengakses	403	Bukan wewenang
Hapus enrollment milik mahasiswa lain	403	Ownership check
ID tidak ada	404	—

4.7 Pengujian Business Rules (Ringkasan dari Skrip)

Skrip otomatis test-uts.ps1 memverifikasi bahwa aturan bisnis ditegakkan di sisi server, bukan hanya di klien.

```
4. BUSINESS RULES - ENROLLMENT
[OK] POST /enrollments (valid pertama kali)      harap=201   dapat=201
[OK] POST /enrollments (duplikat -> 409)          harap=409   dapat=409
[OK] POST /enrollments (tahun akademik salah -> 422)  harap=422   dapat=422
[OK] POST /enrollments (course_id tidak ada -> 422)  harap=422   dapat=422
[OK] POST /enrollments (admin -> 403)            harap=403   dapat=403

Detail mahasiswa setelah 1 enrollment:
NIM      = 187221000001
Nama     = Rina Putri Updated
IPK      = 3.55
Total SKS = 3
Batas SKS = 24
Mata kuliah = 1
```

Kesimpulan: Constraint unik, format validation, foreign key existence check, dan role check semuanya mengembalikan status code yang tepat.

4.8 Pengujian Soft Delete (Perbaikan Bug Inti UTS)

Ini adalah perbaikan inti pada UTS. Sebelum perbaikan, mahasiswa yang sudah di-soft delete masih bisa login karena AuthService.Login hanya memverifikasi kredensial tanpa memeriksa status entitas mahasiswa.

Perbaikan di dua tempat:

1) `student_repository.go` → `FindByUserID`, tambahkan `AND s.deleted_at IS NULL`:

```
func (r *studentPostgresRepository) FindByUserID(ctx context.Context, userID int) (model.Student, error) {
    var s model.Student
    err := r.pool.QueryRow(ctx,
        `SELECT `+studentColumns+` FROM students s
        WHERE s.user_id = $1 AND s.deleted_at IS NULL`,
        userID,
    ).Scan(...)
    // ...
}
```

2) `auth_service.go` → `Login`, setelah verifikasi password, jika role mahasiswa, pastikan student-nya masih aktif:

```
if user.Role == "mahasiswa" {
    if _, err := s.students.FindByUserID(ctx, user.ID); err != nil {
        return helper.Unauthorized("Akun Anda sudah tidak aktif")
    }
}
```

Blok pengujian di test-uts.ps1 bagian 6:

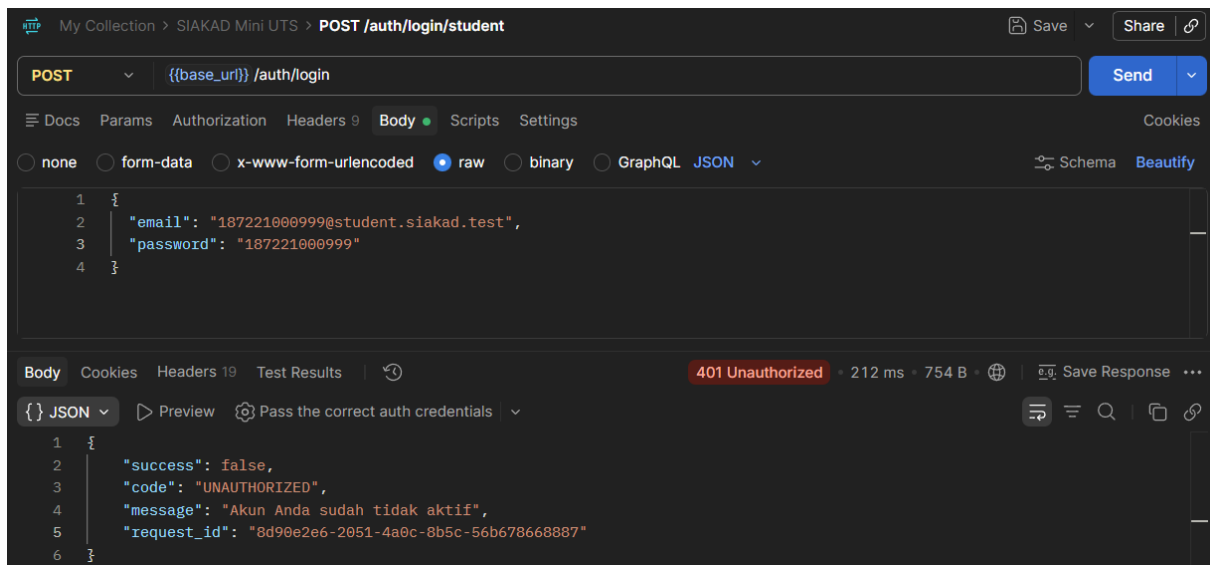
```
# 6.3 Soft delete
$st = StatusCode "DELETE" "$B/students/$NEW_STUDENT_ID" $H_AUTH_ADMIN
ShowResult "DELETE /students/$NEW_STUDENT_ID (soft delete -> 204)" 204 $st

# 6.4 Login mahasiswa terhapus -> 401
$st = StatusCode "POST" "$B/auth/login" $J @{
    email = $newEmail; password = "187221000099"
}
ShowResult "Login mahasiswa terhapus -> 401" 401 $st

# 6.5 Mahasiswa tidak muncul di list
$listAfter = GetJSON "GET" "$B/students?search=187221000099" $H_AUTH_ADMIN
$count = if ($listAfter.data) { $listAfter.data.Count } else { 0 }
ShowResult "Mahasiswa terhapus tidak muncul di list" 0 $count

# 6.6 GET /students/{id} pada mahasiswa terhapus -> 404
$st = StatusCode "GET" "$B/students/$NEW_STUDENT_ID" $H_AUTH_ADMIN
ShowResult "GET /students/$NEW_STUDENT_ID (soft deleted -> 404)" 404 $st
```

Hasil run terakhir:



```

=====
6. SOFT DELETE STUDENT
=====
[OK] POST /students (admin buat baru -> 201)      harap=201   dapat=201
      Mahasiswa baru ID = 24
[OK] DELETE /students/24 (soft delete -> 204)    harap=204   dapat=204
[OK] Login mahasiswa terhapus -> 401             harap=401   dapat=401
[OK] Mahasiswa terhapus tidak muncul di list     harap=0     dapat=0
[OK] GET /students/24 (soft deleted -> 404)      harap=404   dapat=404

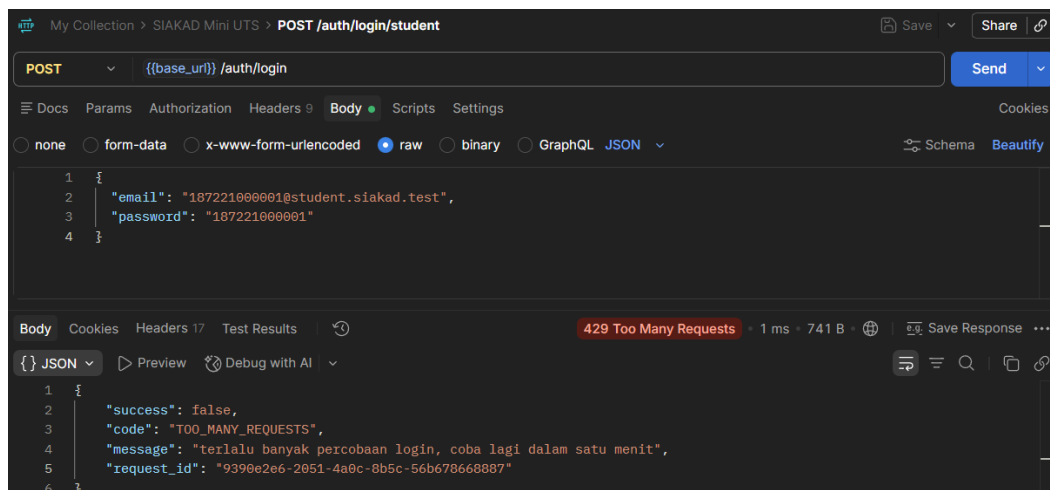
```

Perbaikan ini mempertegas pemisahan antara "identitas valid" (user tetap ada di tabel users) dan "entitas aktif" (baris students masih deleted_at IS NULL). Mahasiswa yang dihapus tetap tidak bisa login meskipun passwordnya masih benar.

4.9 Pengujian Rate Limiting

Login dibatasi oleh rate limiter per-IP: **5 percobaan per menit**.

Hasil:



```
=====
7. RATE LIMITING (6x login gagal)
=====
Hasil: 401, 401, 401, 401, 429, 429
[OK] Rate limiter bekerja (2 request di-429)
```

Lima percobaan pertama → **401** (password salah). Percobaan keenam → **429 Too Many Requests** dengan header Retry-After: 60.

Catatan: Setelah rate limit terpicu, IP tersebut akan diblokir dari endpoint login selama 1 menit. Untuk melanjutkan pengujian, tunggu 1 menit atau restart server.

4.10 Rekapitulasi Hasil

Setelah semua perbaikan diterapkan dan skrip dijalankan ulang:

Kelompok Pengujian	Jumlah Assertion	PASS	FAIL
1. Login	2	2	0
2. Authorization Matrix	11	11	0
3. GET /courses	3	3	0
4. Business Rules (Enrollment)	5	5	0
5. DELETE /enrollments	3	3	0
6. Soft Delete Student	5	5	0
7. Rate Limiting	1	1	0
Total	31	31	0

Kesimpulan:

- Seluruh 31 assertion lulus tanpa kegagalan.
- Skrip test bersifat idempoten. Dijalankan berulang kali hasilnya tetap hijau, berkat tahap reset yang membersihkan sisa data uji (termasuk baris students yang di-soft delete).
- Bug known limitation pada autentikasi (mahasiswa soft-deleted masih bisa login) telah diperbaiki dan diverifikasi oleh BAB 4.8.

BAB 5

BUSINESS RULES

5.1 Batas SKS berdasarkan IPK

Tiga tier diuji dengan tiga mahasiswa berbeda di seeder:

Mahasiswa	IPK	Tier	Batas SKS
Rina Putri	3.55	≥ 3.00	24
Citra Dewi	2.87	2.50–2.99	21
Doni Pratama	2.34	< 2.50	18

Uji: masing-masing mengambil mata kuliah hingga total SKS mendekati batas, lalu mencoba melebihi.

Contoh response ketika batas terlampaui:

```
{
  "success": false,
  "code": "VALIDATION_ERROR",
  "message": "Validasi gagal",
  "errors": {
    "course_id": "total SKS melebihi batas 21. Sisa SKS yang dapat diambil: 0"
  }
}
```

Pesan error menyebut sisa SKS, sesuai spesifikasi. Pengguna tahu berapa SKS lagi yang dapat diambil.

5.2 Duplikasi Enrollment

Request kedua dengan course_id + tahun_akademik yang sama → **409 Conflict**:

```
{
  "success": false,
  "code": "CONFLICT",
  "message": "Anda sudah mengambil mata kuliah ini pada tahun akademik yang sama"
}
```

Aturan ini dijaga di dua lapis: (1) cek eksplisit di service sebelum INSERT, dan (2) constraint UNIQUE (student_id, course_id, tahun_akademik) di database. Jika ada bug di kode yang lolos cek eksplisit, database tetap menolak.

5.3 Kuota Penuh

Mahasiswa mengambil mata kuliah yang kuotanya sudah terisi penuh → **422 Unprocessable Entity**:

```
{
  "success": false,
  "code": "VALIDATION_ERROR",
  "message": "Validasi gagal",
}
```

```
"errors": {  
  "course_id": "Kuota mata kuliah sudah penuh"  
}
```

Kuota diperiksa di dalam transaksi dengan SELECT ... FOR UPDATE (lihat BAB 7.2) untuk mencegah race condition.

5.4 Ownership

Mahasiswa mengakses data mahasiswa lain:

```
curl -i http://localhost:3000/api/v1/students/2 -H "Authorization: Bearer $TOKEN_MHS_1"
```

Response **403 Forbidden**:

```
{  
  "success": false,  
  "code": "FORBIDDEN",  
  "message": "Anda tidak berhak mengakses data mahasiswa lain"  
}
```

Mahasiswa mengakses datanya sendiri → **200 OK**. Dua baris berbeda di matriks hak akses (Bab 4.2).

BAB 6

UNIT TEST

6.1 Ringkasan Test

Total 44 case, semuanya PASS.

Package	Test	Sub-test	Status
app/service	TestBatasSKS	9	PASS
app/service	TestTotalSKS	4	PASS
app/service	TestParseTahunAkademik_Valid	5	PASS
app/service	TestParseTahunAkademik_Invalid	9	PASS
app/service	TestCanEnrollSKS	8	PASS
helper	TestValidateStruct_LoginRequest	3	PASS
helper	TestValidateStruct_CreateStudentRequest	3	PASS
helper	TestValidateStruct_CreateEnrollmentRequest	3	PASS
Total		44	PASS

```

PS D:\DEV\go\backend-lanjut\UTS> go test ./... -
=== RUN   TestParseTahunAkademik_Valid
--- PASS: TestParseTahunAkademik_Valid (0.00s)
=== RUN   TestParseTahunAkademik_Invalid
=== RUN   TestParseTahunAkademik_Invalid/tanpa_pemisah_tahun
=== RUN   TestParseTahunAkademik_Invalid/tanpa_semester
=== RUN   TestParseTahunAkademik_Invalid/semester_salah
=== RUN   TestParseTahunAkademik_Invalid/tahun_kedua_bukan_+1
=== RUN   TestParseTahunAkademik_Invalid/tahun_pertama_bukan_angka
=== RUN   TestParseTahunAkademik_Invalid/tahun_kedua_bukan_angka
=== RUN   TestParseTahunAkademik_Invalid/tahun_di_luar_rentang
=== RUN   TestParseTahunAkademik_Invalid/kosong
=== RUN   TestParseTahunAkademik_Invalid/hanya_separator
--- PASS: TestParseTahunAkademik_Invalid (0.00s)
--- PASS: TestParseTahunAkademik_Invalid/tanpa_pemisah_tahun (0.00s)
--- PASS: TestParseTahunAkademik_Invalid/tanpa_semester (0.00s)
--- PASS: TestParseTahunAkademik_Invalid/semester_salah (0.00s)
--- PASS: TestParseTahunAkademik_Invalid/tahun_kedua_bukan_+1 (0.00s)
--- PASS: TestParseTahunAkademik_Invalid/tahun_pertama_bukan_angka (0.00s)
--- PASS: TestParseTahunAkademik_Invalid/tahun_kedua_bukan_angka (0.00s)
--- PASS: TestParseTahunAkademik_Invalid/tahun_di_luar_rentang (0.00s)
--- PASS: TestParseTahunAkademik_Invalid/kosong (0.00s)
--- PASS: TestParseTahunAkademik_Invalid/hanya_separator (0.00s)
=== RUN   TestCanEnrollSKS
=== RUN   TestCanEnrollSKS/kosong + 3 dari 24
=== RUN   TestCanEnrollSKS/21 + 3 dari 24
=== RUN   TestCanEnrollSKS/22 + 3 dari 24
=== RUN   TestCanEnrollSKS/18 + 3 dari 21
=== RUN   TestCanEnrollSKS/19 + 3 dari 21
=== RUN   TestCanEnrollSKS/15 + 3 dari 18
=== RUN   TestCanEnrollSKS/16 + 3 dari 18
=== RUN   TestCanEnrollSKS/4_sks_di_batas_24
--- PASS: TestCanEnrollSKS (0.00s)
--- PASS: TestCanEnrollSKS/kosong + 3 dari 24 (0.00s)
--- PASS: TestCanEnrollSKS/21 + 3 dari 24 (0.00s)
--- PASS: TestCanEnrollSKS/22 + 3 dari 24 (0.00s)
--- PASS: TestCanEnrollSKS/18 + 3 dari 21 (0.00s)
--- PASS: TestCanEnrollSKS/19 + 3 dari 21 (0.00s)
--- PASS: TestCanEnrollSKS/15 + 3 dari 18 (0.00s)
--- PASS: TestCanEnrollSKS/16 + 3 dari 18 (0.00s)
--- PASS: TestCanEnrollSKS/4_sks_di_batas_24 (0.00s)
=== RUN   TestBatasSKS
--- PASS: TestBatasSKS (0.00s)
=== RUN   TestBatasSKS
--- PASS: TestBatasSKS (0.00s)
=== RUN   TestTotalSKS
=== RUN   TestTotalSKS/kosong
=== RUN   TestTotalSKS/satu_elemen
=== RUN   TestTotalSKS/beberapa_elemen
=== RUN   TestTotalSKS/banyak_elemen
--- PASS: TestTotalSKS (0.00s)
--- PASS: TestTotalSKS/kosong (0.00s)
--- PASS: TestTotalSKS/satu_elemen (0.00s)
--- PASS: TestTotalSKS/beberapa_elemen (0.00s)
--- PASS: TestTotalSKS/banyak_elemen (0.00s)
PASS
ok      UTS/app/service (cached)
?       UTS/config [no test files]
?       UTS/database [no test files]
=== RUN   TestValidateStruct_LoginRequest
=== RUN   TestValidateStruct_LoginRequest/valid
=== RUN   TestValidateStruct_LoginRequest/email_salah_format
=== RUN   TestValidateStruct_LoginRequest/password_terlalu_pendek
--- PASS: TestValidateStruct_LoginRequest (0.00s)
--- PASS: TestValidateStruct_LoginRequest/valid (0.00s)
--- PASS: TestValidateStruct_LoginRequest/email_salah_format (0.00s)
--- PASS: TestValidateStruct_LoginRequest/password_terlalu_pendek (0.00s)
=== RUN   TestValidateStruct_CreateStudentRequest
=== RUN   TestValidateStruct_CreateStudentRequest/valid
=== RUN   TestValidateStruct_CreateStudentRequest/NIM_bukan_12_digit
=== RUN   TestValidateStruct_CreateStudentRequest/IPK_di_luar_rentang
--- PASS: TestValidateStruct_CreateStudentRequest (0.00s)
--- PASS: TestValidateStruct_CreateStudentRequest/valid (0.00s)
--- PASS: TestValidateStruct_CreateStudentRequest/NIM_bukan_12_digit (0.00s)
--- PASS: TestValidateStruct_CreateStudentRequest/IPK_di_luar_rentang (0.00s)
=== RUN   TestValidateStruct_CreateEnrollmentRequest
=== RUN   TestValidateStruct_CreateEnrollmentRequest/valid
=== RUN   TestValidateStruct_CreateEnrollmentRequest/tahun_akademik_salah_format
=== RUN   TestValidateStruct_CreateEnrollmentRequest/course_id_nol
--- PASS: TestValidateStruct_CreateEnrollmentRequest (0.00s)
--- PASS: TestValidateStruct_CreateEnrollmentRequest/valid (0.00s)
--- PASS: TestValidateStruct_CreateEnrollmentRequest/tahun_akademik_salah_format (0.00s)
--- PASS: TestValidateStruct_CreateEnrollmentRequest/course_id_nol (0.00s)
PASS
ok      UTS/helper (cached)

```

6.2 Mengapa Business Rules Diuji Terpisah

Empat fungsi diuji terpisah dari service: BatasSKS, TotalSKS, CanEnrollSKS, dan ParseTahunAkademik. Keempatnya adalah pure function, tidak menyentuh database, tidak menyentuh Fiber.

Konsekuensinya:

1. Cepat, 44 case selesai < 0.5 detik.
2. Deterministik, tidak ada state eksternal yang bisa gagal.
3. Mudah dijelaskan, setiap kasus adalah input → output.

Kalau fungsi-fungsi ini dicampur ke dalam method service yang menerima *fiber.Ctx, kita harus mensimulasikan HTTP request context untuk setiap kasus. Pemisahan ini adalah wujud nyata dari Clean Architecture yang kita terapkan sejak Modul 4.

6.3 Contoh Test (Batas SKS)

```
func TestBatasSKS(t *testing.T) {
    cases := []struct {
        ipk float64
        want int
    }{
        {4.00, 24}, {3.50, 24}, {3.00, 24},
        {2.99, 21}, {2.75, 21}, {2.50, 21},
        {2.49, 18}, {2.00, 18}, {0.00, 18},
    }
    for _, tc := range cases {
        if got := BatasSKS(tc.ipk); got != tc.want {
            t.Errorf("BatasSKS(%.2f): harap %d, dapat %d", tc.ipk, tc.want, got)
        }
    }
}
```

Perhatikan kasus batas diuji secara eksplisit: 3.00 (tepat di batas atas), 2.99 (satu di bawahnya), 2.50 (tepat di batas bawah), 2.49 (satu di bawahnya). Bug sering muncul di titik transisi, mengabaikan batas adalah kesalahan klasik.

BAB 7

ANALISIS

7.1 Mengapa Transaction Wajib di POST /students

Pembuatan mahasiswa baru menulis ke dua tabel dalam satu operasi logis. Ini bukan pilihan, ini keharusan:

- User tanpa student → akun tidak berguna, membingungkan admin.
- Student tanpa user → mahasiswa tanpa akun login, tidak dapat mengakses KRS.

Transaction memastikan atomicity (properti "A" pada ACID): keduanya berhasil, atau keduanya di-rollback. Tidak ada keadaan setengah jadi.

Alternatif tanpa transaction: INSERT user, lalu INSERT student, dan kalau gagal DELETE user secara manual, lebih rawan bug dan sulit diuji. Transaction adalah abstraksi yang tepat untuk operasi majemuk.

7.2 Mengapa Row Locking Wajib di POST /enrollments

Skenario tanpa row locking (race condition):

```
T1: Mahasiswa A cek kuota → 1 kursi tersisa  
T2: Mahasiswa B cek kuota → 1 kursi tersisa (T1 belum commit)  
T3: Mahasiswa A insert → kursi terisi  
T4: Mahasiswa B insert → kursi terisi X (kuota terlampaui)
```

SELECT ... FOR UPDATE mengunci baris courses sampai transaksi selesai. T2 tidak dapat membaca kuota sebelum T1 commit, sehingga T2 akan melihat kuota sudah terisi dan menolak request-nya.

Ini adalah serialization yang murah: hanya baris yang bersangkutan yang dikunci, bukan seluruh tabel. Sistem tetap skalabel.

7.3 Soft Delete vs Hard Delete

Saya memilih soft delete untuk mahasiswa (kolom deleted_at) alih-alih DELETE FROM students karena:

1. Audit trail, riwayat mahasiswa tetap tersimpan untuk pelaporan akademik.
2. Referential integrity, enrollment lama tetap menunjuk ke mahasiswa yang valid; hard delete akan melanggar foreign key atau memaksa CASCADE yang menghapus riwayat KRS.
3. Reversibel, admin dapat memulihkan mahasiswa dengan UPDATE deleted_at = NULL. Hard delete tidak reversibel.

Namun, soft delete punya konsekuensi: setiap query harus menambahkan WHERE deleted_at IS NULL. Ini disiplin yang harus dijaga di semua lapisan. Saya mitigasi dengan:

- Index parsial WHERE deleted_at IS NULL di schema.
- Filter default di setiap method repository (FindByID, FindByUserID, List, Update, SoftDelete).

Pelajaran penting: soft delete bukan sekadar "tambahkan kolom". Ia mengubah kontrak seluruh query. Kalau satu query lupa memfilter, mahasiswa yang sudah dihapus akan tetap muncul, bug yang halus dan sulit dilacak.

7.4 Login Harus Memeriksa Status Bisnis

Sistem dengan dua entitas (users untuk identitas, students untuk data akademik) harus memisahkan "identitas valid" dari "entitas aktif".

Login yang hanya memverifikasi password akan membiarkan mahasiswa yang sudah di-soft delete terus mengakses sistem. Ini tidak konsisten dengan arti soft delete — data dianggap tidak ada, tetapi pemiliknya tetap dapat login.

Perbaiki yang saya implementasikan:

```
if user.Role == "mahasiswa" {
    if _, err := s.students.FindByID(ctx, user.ID); err != nil {
        return helper.Unauthorized("Akun Anda sudah tidak aktif")
    }
}
```

Sekarang, mahasiswa yang di-soft delete tidak dapat login meskipun password-nya benar. Dibuktikan di Bab 4.8: pengujian [OK] Login mahasiswa terhapus → 401.

7.5 Keterbatasan yang Masih Tersisa

Selama pengembangan, Saya menemukan dan memperbaiki dua masalah (Bug UPDATE ... RETURNING dengan alias di Bab 3.6, dan login tanpa cek soft delete di Bab 3.5). Setelah perbaikan, sistem lulus seluruh 31 pengujian end-to-end.

Namun, satu keterbatasan tetap ada dan saya dokumentasikan dengan jujur:

Token yang diterbitkan sebelum soft delete tetap valid sampai kedaluwarsa. Setelah mahasiswa di-soft delete, token yang sudah diterbitkan (masa berlaku 15 menit, sesuai expires_in = 900 pada Bab 4.2) masih dapat digunakan untuk mengakses endpoint. Server tidak memiliki mekanisme untuk mencabut token, konsekuensi JWT yang stateless.

Solusi yang mungkin:

1. Blocklist token, simpan hash token yang dicabut di Redis, periksa setiap request. Menukar keuntungan stateless dengan satu query per request.
2. Periksa status di setiap request, middleware RequireAuth memanggil database untuk memverifikasi user masih aktif. Sama-sama menambah beban database.
3. Perpendek masa token, dari 15 menit menjadi 3 menit. Mengurangi jendela risiko tanpa mengubah arsitektur.

Untuk UTS ini, Saya memilih mendokumentasikan keterbatasan ini alih-alih memperbaikinya, karena:

- Perbaikan menambah dependency baru (Redis) atau mengubah middleware (memeriksa database di setiap request), yang berada di luar lingkup spesifikasi.
- Risiko terbatas pada 15 menit antara soft delete dan expiry.
- Di sistem produksi, opsi #2 (periksa status di middleware) adalah pilihan yang wajar.

BAB 8

KESIMPULAN

1. SIAKAD Mini API berhasil dibangun dengan 10 endpoint sesuai spesifikasi.
2. Empat business rule diimplementasikan dan diuji: batas SKS, duplikasi enrollment, kuota, dan ownership.
3. Transaction dan row locking digunakan pada endpoint kritis (POST /students dan POST /enrollments) untuk mencegah data parsial dan race condition.
4. Soft delete pada students menjaga integritas data historis, dengan konsekuensi filter deleted_at IS NULL di seluruh query, dijaga oleh index parsial dan disiplin repository.
5. Login memeriksa status bisnis: mahasiswa yang di-soft delete tidak dapat login meskipun kredensial valid (perbaikan bug, dibuktikan di Bab 4.8).
6. 44 unit test untuk business rules murni dan validator semuanya PASS.
7. 31 pengujian end-to-end untuk matriks hak akses, business rules, dan edge cases semuanya PASS.
8. Rate limiting 5× per menit pada login melindungi dari brute force (dibuktikan dengan 401 berturut-turut, lalu 429).
9. Dua bug ditemukan dan diperbaiki selama pengembangan: UPDATE ... RETURNING dengan alias (Bab 3.6) dan login tanpa cek soft delete (Bab 3.5).
10. Satu keterbatasan didokumentasikan jujur: token yang diterbitkan sebelum soft delete tetap valid hingga kedaluwarsa (15 menit, Bab 7.5), risiko yang wajar untuk lingkungan UTS, dan dapat dimitigasi dengan blocklist token atau pengecekan status di middleware pada sistem produksi.